COMPUTER SCIENCE AND ARCHITECTURE
NASM32 Calculator
RICOHERMOSO, RAYMOND JR.

LABEXER1

Create ASM using NASM32, that implements procedure

MENU

[A]dd

[S]ubtract

[M]ultiply

[D]ivide

Enter your choice:

Enter first number: 2

Enter second number: 3

The sum is 5

Do you want to try again (Y/N)

if yes loop back to the menu (ADD clear screen)

If no Terminate the program

Add Test cases - include the screenshots (3 per operation)

Explain how the program works

extra points if two digit input.

add compilation process for each test cases

add git repo for the activity

SOURCE CODE WITH EXPLANATIONS:

```
; COMPUTER SCIENCE AND ARCHITECTURE
; NASM32 Calculator
; RICOHERMOSO, RAYMOND JR.

section .data ; this part is where the text is displayed and where the user inputs integers and also reserves bytes.
    CLR db 27, "[2J", 27, "[1;1H", 0
    MENU db "----------------------------------", 10, "MENU", 10, "[A]dd", 10, "[S]ubtract", 10, "[M]ultiply", 10, "[D]ivide", 10,
"----------------------------------", 10, "Choice: ", 0
    P1 db "Enter first number: ", 0
    P2 db "Enter second number: ", 0
    RES db "The result is ", 0
    REM db " (remainder ", 0
    ERR db "Invalid Negative input!", 10, 0
    AGAIN db "Try again (Y/N): ", 0

section .bss
    buf resb 32
    num1 resd 1
    num2 resd 1
    op resb 2

section .text
    global _start

_start:
menu:
    mov esi, CLR
    call print
    mov esi, MENU
    call print
    call read
    mov al, [buf]
    and al, 0xDF
    mov [op], al

    mov esi, P1
    call print
    call read
    call atoi
    jc err_out
    mov [num1], eax

    mov esi, P2
    call print
    call read
    call atoi
    jc err_out
    mov [num2], eax

    mov eax, [num1]
    mov ebx, [num2]
    mov cl, [op]

    cmp cl, 'A'
    je do_add
    cmp cl, 'S'
    je do_sub
    cmp cl, 'M'
    je do_mul
    cmp cl, 'D'
```

```
    je do_div
    jmp retry

do_add:
    add eax, ebx
    jmp print_res
do_sub:
    sub eax, ebx
    jl err_out      ; this prevents any negative result
    jmp print_res
do_mul:
    imul eax, ebx
    jmp print_res
do_div:
    cmp ebx, 0
    je err_out
    xor edx, edx
    div ebx
    push edx

    push eax
    mov esi, RES
    call print
    pop eax
    call print_num

    pop edx
    cmp edx, 0
    je done_div
    push edx
    mov esi, REM
    call print
    pop eax
    call print_num
    mov byte [buf], ')'
    mov byte [buf+1], 0
    mov esi, buf
    call print
done_div:
    mov byte [buf], 10
    mov byte [buf+1], 0
    mov esi, buf
    call print
    jmp retry

print_res:
    push eax
    mov esi, RES
    call print
    pop eax
    call print_num
    mov byte [buf], 10
    mov byte [buf+1], 0
    mov esi, buf
    call print
    jmp retry

err_out:
    mov esi, ERR
    call print

retry:
    mov esi, AGAIN
    call print
    call read
```

```
    mov al, [buf]
    and al, 0xDF
    cmp al, 'Y'
    je menu

    mov eax, 1
    xor ebx, ebx
    int 0x80

; --- HELPERS ---
print:
    pusha
    mov edx, esi
.l: cmp byte [edx], 0
    je .p
    inc edx
    jmp .l
.p: sub edx, esi
    mov ecx, esi
    mov eax, 4
    mov ebx, 1
    int 0x80
    popa
    ret

read:
    mov eax, 3
    mov ebx, 0
    mov ecx, buf
    mov edx, 32
    int 0x80
    ret

atoi:
    xor eax, eax
    mov esi, buf
.l: mov bl, [esi]
    cmp bl, '-'     ; this rejects negative sign
    je .err
    cmp bl, '0'
    jl .done
    cmp bl, '9'
    jg .done
    sub bl, '0'
    imul eax, 10
    movzx ebx, bl
    add eax, ebx
    inc esi
    jmp .l
.err:
    stc             ; this sets carry flag to signal error
    ret
.done:
    clc             ; this clears  carry flag on success
    ret

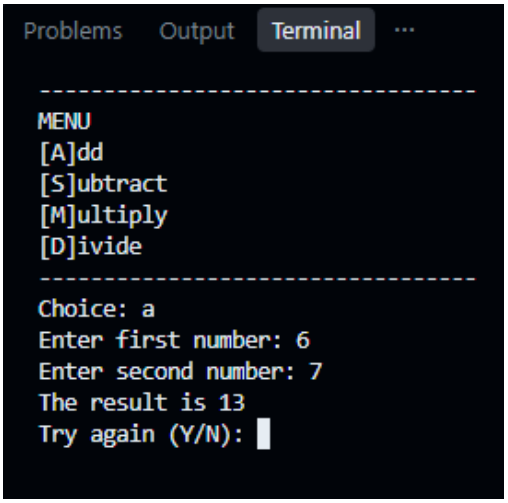
print_num:
    pusha
    mov edi, buf + 31
    mov byte [edi], 0
    mov ebx, 10
    test eax, eax
    jnz .c
    dec edi
```

```
    mov byte [edi], '0'
    jmp .p
.c: xor edx, edx
    div ebx
    add dl, '0'
    dec edi
    mov [edi], dl
    test eax, eax
    jnz .c
.p: mov esi, edi
    call print
    popa
    ret
```

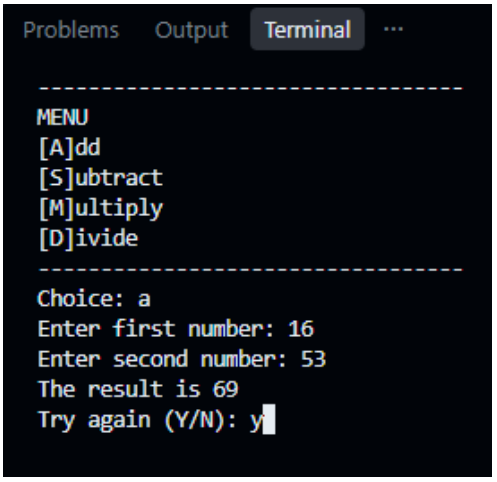
TEST CASES:

ADDITION:

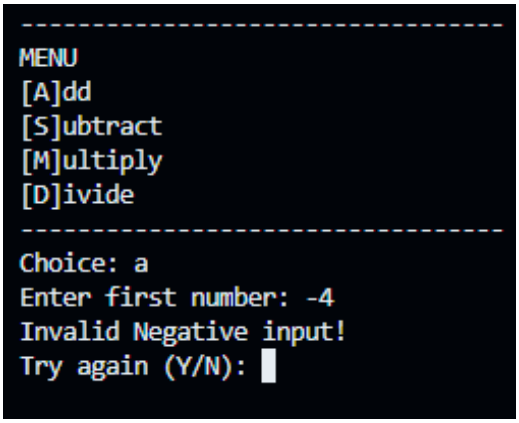
Single digit input:



Double digit input:



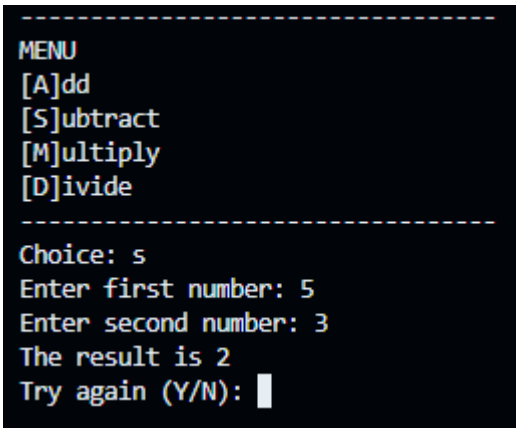
Negative input:



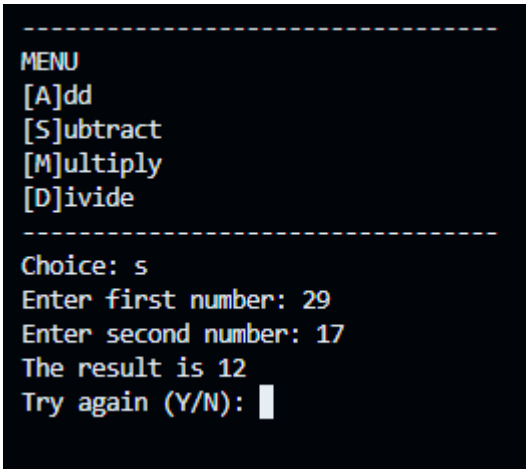
Inputting a negative number calls line 153 and rejects negative sign

SUBTRACTION:

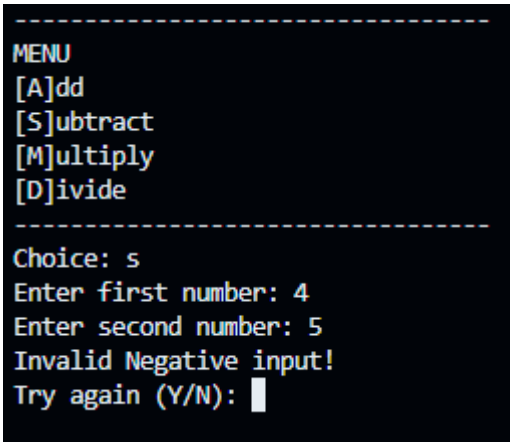
Single digit input:



Double digit input:



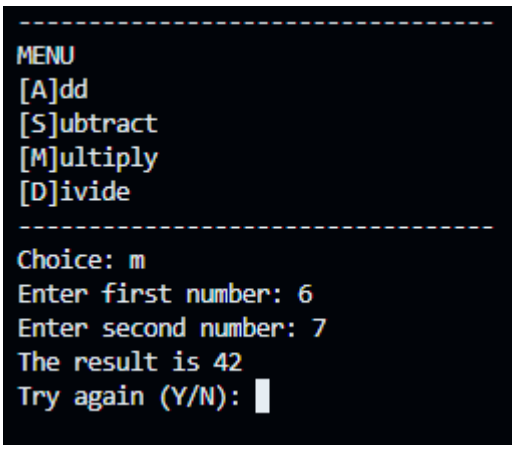
Negative output:



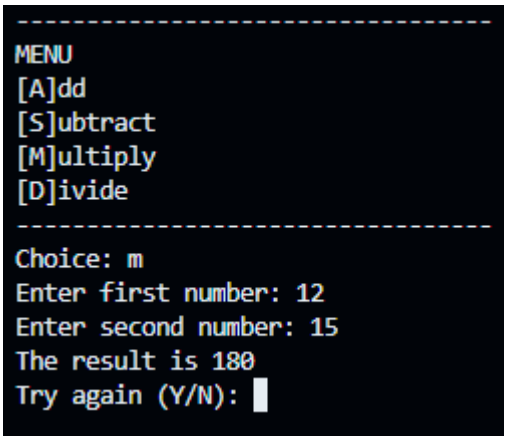
if the result turns out to be negative, the program automatically logs it and consider as invalid.

MULTIPLICATION:

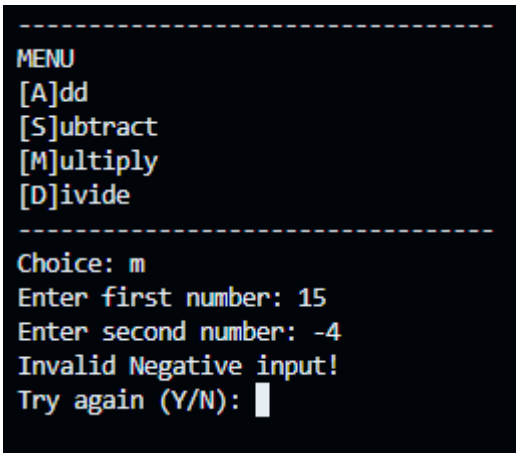
Single digit input:



Double digit input:

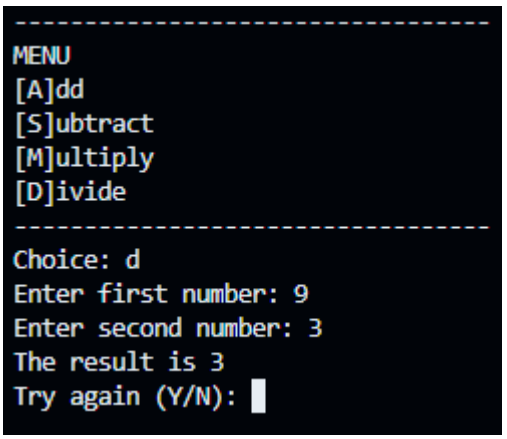


Negative integer input:

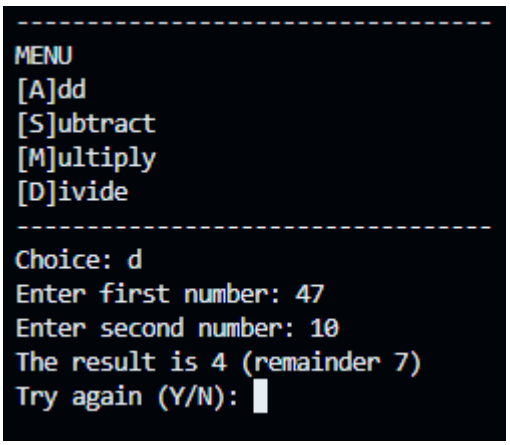


DIVISION:

Single digit input:



Double digit input:



Zero input:

